

ARM 汇编指令 · 完整总结

i.MX6ULL (Cortex-A7 / ARMv7) | 32 位 ARM 指令集

一、立即数

立即数是直接编码在指令中的常数值。ARM 指令为 32 位固定长度，其中立即数编码仅占 12 位：8 位无符号数 + 4 位旋转位。

字段	位数	含义
8位立即数	8 bit	无符号数 0~255
旋转位	4 bit	将8位数循环右移（旋转位×2）次

- 合法立即数 = 8 位无符号数 偶数次循环右移 得到的 32 位值
- 不是所有 32 位数都能做立即数（如 0x101 不合法）
- MVN 指令：将立即数按位取反后装入寄存器，扩展了立即数范围

立即数判断示例：0xFA00

```
0xFA00 = 0x0000FA00
        : 0000 0000 0000 0000 1111 1010 0000 0000

0xFA = 11111010 (8 位)
      24  = 12 (12×2=24)
→ 0xFA00

mov r0, #0xFA00 //
mvn r0, #0 // r0 = 0xFFFFFFFF ( )
```

二、跳转指令：b / bl / bx

指令	全称	保存返回地址	典型用途
b	Branch	否	无条件跳转（类似 goto），跳转后不返回
bl	Branch with Link	是，自动保存到 LR	bx lr
bx	Branch and Exchange	否，但可恢复 PC	bx lr ARM/Thumb

函数调用流程：

```
// C : sum = add(3, 5)
mov r0, #3 // a=3
mov r1, #5 // b=5
bl add //    add        LR
// add    ...
bx lr //    LR    PC
```

三、栈的分类与 ARM 栈类型

3.1 栈的四个基本概念

概念	定义	操作方式
空栈	SP 指向空位（无有效数据），先写数据再移指针	*sp++ *sp--
满栈	SP 指向已有数据，先移指针再写新数据	*++sp *--sp
增栈	入栈时 SP 向高地址方向增长（SP 值增大）	SP 递增
减栈	入栈时 SP 向低地址方向递减（SP 值减小）	SP 递减

3.2 四种栈类型

栈类型	缩写	组合	入栈操作	出栈操作
空减栈	ED	空 + 减	先写数据，SP 减小	先取数据，SP 增大
空增栈	EA	空 + 增	先写数据，SP 增大	先取数据，SP 减小
满减栈	FD	满 + 减	SP 先减小，再写数据	先取数据，SP 再减小
满增栈	FA	满 + 增	SP 先增大，再写数据	先取数据，SP 再增大

3.3 ARM 采用的栈类型

ARM 处理器采用 满减栈（FD, Full Descending）：

- 满：SP 指向的位置有有效数据，入栈时先减小 SP 再写入
- 减：入栈时 SP 向低地址方向递减（从 0x40001000 向 0x40000000 生长）
- 对应指令：stmfd（入栈）和 ldmfd（出栈）

内存布局示意：

```
0x40001000 ← SP
0x4000FFC ←   SP   SP
...
0x40000000 ←
```

四、内存访问指令：ldr / str

指令	全称	作用	数据方向
ldr	Load Register	从 RAM 加载数据到通用寄存器	内存 → 寄存器
str	Store Register	将寄存器值写入 RAM	寄存器 → 内存

核心场景：

- 加载非立即数（立即数放不下的 32 位常量通过 ldr 伪指令从内存读取）
- 读写外设寄存器（GPIO、UART 等映射到内存地址空间的寄存器）
- 变量读写（C 语言变量分配在 RAM 中，汇编通过 ldr/str 访问）

示例:

```
// C: int i = 2; int j = 3; sum = i + j;
// i@0x40000000, j@0x40000004, sum@0x40000008

ldr r0, =0x40000000 // i
ldr r1, [r0] // r1 = i = 2

ldr r0, =0x40000004 // j
ldr r2, [r0] // r2 = j = 3

add r3, r1, r2 // r3 = 2 + 3 = 5

ldr r0, =0x40000008 // sum
str r3, [r0] // sum = 5
```

五、多寄存器入栈/出栈: stmfid / ldmfd

指令	全称	作用	说明
stmfd	Store Multiple Full Descending	将多个寄存器值压入栈（入栈）	SP 先减小，再依次写入各寄存器值
ldmfd	Load Multiple Full Descending	从栈恢复多个寄存器值（出栈）	依次读取并恢复各寄存器

函数调用中的上下文保存与恢复:

```
// — —
stmfd sp!, {r0-r12, lr} // r0~r12 lr

// — —
// ... ...

// — —
ldmfd sp!, {r0-r12, pc} // r0~r12 lr→pc
```

技巧: 出栈时用 pc 替代 lr, 一条 ldmfd 同时完成寄存器恢复和函数返回。stmfd 中 ! 表示写回 SP (更新栈指针)。

六、位操作指令: bic / orr

指令	全称	作用	操作
bic	Bit Clear	将操作数中指定 bit 位清 0	$Rd = Rn \text{ AND NOT}(op2)$
orr	OR	将操作数中指定 bit 位置 1	$Rd = Rn \text{ OR } op2$

典型应用: 修改 CPSR 切换工作模式 (如切换到 USER 模式 0b10000)

```
// 1: CPSR r0
mrs r0, cpsr

// 2: 5 (bic 0)
bic r0, r0, #0x1F // 0b11111 → 0b00000

// 3: USER (orr 1)
```

```
orr r0, r0, #0x10 // 0b10000 = USER

// 4: CPSR
msr cpsr, r0
```

模式	编码 [4:0]	bic 后 orr 的值
USER	10000	orr r0, r0, #0x10
FIQ	10001	orr r0, r0, #0x11
IRQ	10010	orr r0, r0, #0x12
SVC	10011	orr r0, r0, #0x13
Abort	10111	orr r0, r0, #0x17
Undefined	11011	orr r0, r0, #0x1B
System	11111	orr r0, r0, #0x1F

七、特殊寄存器读写：mrs / msr

指令	全称	作用	方向
mrs	Move from Special Register	CPSR/SPSR	特殊 → 通用（读）
msr	Move to Special Register	CPSR/SPSR	通用 → 特殊（写）

ARM 无法直接修改 CPSR，必须遵循"读 → 改 → 写"三步流程：

```
mrs r0, cpsr // 1. CPSR → r0
bic r0, r0, #0x1F // 2.
orr r0, r0, #0x13 // SVC
msr cpsr, r0 // 3. r0 → CPSR
```

八、汇编与 C 语言互调

8.1 汇编调用 C 函数

参数个数	传递方式	返回值
≤ 4 个	R0~R3 1 →R0 2→R1 3→R2 4→R3	C R0
> 4 个	4 R0~R3	C R0

```
// C: int add(int a, int b) { return a + b; }

//      :
mov r0, #3 // a=3 → R0
mov r1, #5 // b=5 → R1
bl add // add
//      R0 (R0=8)
```

8.2 C 调用汇编函数

C 语言调用汇编时，C 编译器自动将参数放入 R0~R3，汇编函数直接从寄存器取参数，结果通过 R0 返回。

```
// C :
extern int asm_add(int a, int b);
int sum = asm_add(3, 5); // C      3→R0, 5→R1

//      :
asm_add:
add r0, r0, r1 // R0 = R0 + R1 = 3 + 5 = 8
bx lr //      R0
```

互调传参要点：

- R0~R3 用于传参和返回值，是 ARM ATPCS 调用约定的核心规则
- 超过 4 个参数时必须用栈，注意参数压栈顺序与出栈顺序严格对应
- R4~R11 是被调用者保存寄存器（callee-saved），汇编函数使用前必须压栈保存
- R0~R3 是调用者保存寄存器（caller-saved），调用方需自行保存

九、指令后缀 s 与条件执行

ARM 指令后加 s 后缀表示 ALU 运算结果影响 CPSR 中的 NZCV 标志位：

标志位	含义	置位条件
N	Negative 负数标志	结果最高位为1（负数）
Z	Zero 零标志	结果为0
C	Carry 进位标志	加法有进位或减法无借位
V	Overflow 溢出标志	有符号数运算溢出

```
mov r0, #3
cmp r0, #5 //   r0 5      NZCV
//   N=1, Z=0 (3 < 5)

movt r1, #0 //   s      NZCV
subs r2, r0, #3 //   s r2=r0-3  NZCV
```

十、ARM 汇编指令速查表

指令	功能	示例	说明
mov	立即数→寄存器	mov r0, #3	只能传立即数或寄存器间赋值
mvn	取反后装入	mvn r0, #0	r0 = 0xFFFFFFFF
b	无条件跳转	b label	不保存返回地址
bl	带链接跳转	bl func	保存返回地址到LR

指令	功能	示例	说明
bx	寄存器跳转	bx lr	用于函数返回
ldr	内存→寄存器	ldr r0, [r1]	r1
str	寄存器→内存	str r0, [r1]	r0 r1
stmfd	多寄存器入栈	stmfd sp!, {r0-r3, lr}	ARM满减栈入栈
ldmfd	多寄存器出栈	ldmfd sp!, {r0-r3, pc}	ARM满减栈出栈
bic	位清零	bic r0, r0, #0x1F	清除低5位
orr	位置一	orr r0, r0, #0x10	bit4置1
mrs	读特殊寄存器	mrs r0, cpsr	CPSR→r0
msr	写特殊寄存器	msr cpsr, r0	r0→CPSR
cmp	比较	cmp r0, r1	r0-r1 NZCV
add	加法	add r0, r1, r2	r0=r1+r2
sub	减法	sub r0, r1, r2	r0=r1-r2